Experiment 1

Aim : NumPy, Pandas, Matplotlib & Seaborn for Machine Learning

Dataset :

https://drive.google.com/file/d/1BAmvitfDeRbdHeJMBRciKAIXbmkGg9cN/view?usp=sharing

Theory :

PART 1: GENERAL THEORY

1. NumPy (Numerical Python) NumPy is the fundamental package for scientific computing in Python. It provides a high-performance multidimensional array object. While standard Python lists can hold different types of data, NumPy arrays are designed for math. They are much faster and use less memory.

In this lab, we use NumPy for:

- Statistical calculations: Computing the average (mean), the middle value (median), and the spread of data (standard deviation).
- Vectorization: Performing a single operation (like multiplying by 2) on an entire column of data at once instead of using a slow loop.
- Normalization: Using the Min-Max formula to scale scores between 0 and 1 so different datasets can be compared fairly.

2. Pandas (Python Data Analysis Library) Pandas is the primary tool for data manipulation. It introduces the DataFrame, which is essentially a powerful, programmable spreadsheet.

In this lab, we use Pandas for:

- Data Loading: Reading the CSV file into a format the computer can understand.
- Data Cleaning: Identifying missing information (null values) and handling them.
- Feature Engineering: Creating new columns, such as the Performance Label (Good, Average, Low), based on the raw Final Score.

3. Data Visualization (Matplotlib and Seaborn) Visualization is the process of turning numbers into pictures to spot trends that are invisible in a table.

- Matplotlib: This is the "grandfather" library of Python plotting. It gives you total control over every line and label. We use it for basic Line Plots and Histograms.
- Seaborn: Built on top of Matplotlib, Seaborn is designed for statistical art. It makes complex charts like Heatmaps and Boxplots with very little code.

PART 2: DATASET THEORY (STUDENT PERFORMANCE)

The Student Performance dataset is a synthetic (generated) dataset designed to simulate real-world academic patterns. Understanding the relationship between these variables is the core of "Dataset Theory."

1. The Independent Variables (Causes)
- Hours Studied: This is usually the strongest predictor of success. In our data generation, we assume that more hours lead to higher scores.
- Attendance: This represents the student's consistency. High attendance often correlates with higher engagement and better grades.
2. The Dependent Variable (Effect)
- Final Score: This is the outcome we are measuring. It ranges from 0 to 100. It is influenced by the study hours and attendance, plus a bit of "random noise" to represent things like luck or test difficulty.
3. Correlation and Distribution
- Correlation: We use a Heatmap to see how much one variable affects another. A positive correlation between Hours and Score means the "more you study, the more you earn."
- Distribution: We use a Histogram to see the spread of scores. It tells us if the test was "easy" (most scores on the right) or "hard" (most scores on the left).
- Categorical Analysis: By using a Boxplot, we group students by their Performance category. This allows us to see the "typical" score for a Good student versus an Average student and identify any unusual outliers.

Code :

#Load Dataset :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('student_performance.csv')
```

 # Exercise 1: NumPy Basics

```
print("--- Exercise 1: NumPy Basics ---")

# 1. Load Final_Score as NumPy array
final_scores = df['Final_Score'].to_numpy()

# 2. Compute mean, median, and standard deviation
mean_val = np.mean(final_scores)
median_val = np.median(final_scores)
```

```
std_val = np.std(final_scores)

print(f"Mean Final Score: {mean_val:.2f}")
print(f"Median Final Score: {median_val:.2f}")
print(f"Standard Deviation: {std_val:.2f}")

# 3. Perform Min-Max normalization
# Formula: (x - min) / (max - min)
min_val = np.min(final_scores)
max_val = np.max(final_scores)
normalized_scores = (final_scores - min_val) / (max_val - min_val)

print("Normalized Scores (first 5):", np.round(normalized_scores[:5], 2))
print("\n")
```

 # Exercise 2: Pandas Data Handling

```
print("--- Exercise 2: Pandas Data Handling ---")

# 1. Load CSV (Already done above)

# 2. Check shape, columns, and missing values
print(f"Shape of dataset: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
print("Missing Values:\n", df.isnull().sum())

# 3. Create Performance label based on Final_Score
# Criteria: >= 75 (Good), >= 60 (Average), < 60 (Low)
def classify_performance(score):
    if score >= 75:
        return 'Good'
    elif score >= 60:
        return 'Average'
    else:
        return 'Low'

df['Performance'] = df['Final_Score'].apply(classify_performance)
print("\nData with Performance Label (first 5 rows):")
print(df[['Final_Score', 'Performance']].head())
print("\n")
```

 # Exercise 3: Matplotlib Visualization

```
print("--- Exercise 3: Matplotlib Visualization ---")
```

```
# 1. Line plot: Hours_Studied vs Final_Score
# We sort by Hours_Studied to make the line plot look clean
df_sorted = df.sort_values(by='Hours_Studied')

plt.figure(figsize=(8, 5))
plt.plot(df_sorted['Hours_Studied'], df_sorted['Final_Score'], marker='o', linestyle='-', color='blue')
plt.title('Line Plot: Hours Studied vs Final Score')
plt.xlabel('Hours Studied')
plt.ylabel('Final Score')
plt.grid(True)
plt.show()

# 2. Histogram of Final_Score
plt.figure(figsize=(8, 5))
plt.hist(df['Final_Score'], bins=10, color='skyblue', edgecolor='black')
plt.title('Histogram of Final Score')
plt.xlabel('Final Score')
plt.ylabel('Frequency')
plt.show()


# Exercise 4: Seaborn Visualization

print("--- Exercise 4: Seaborn Visualization ---")

# 1. Scatter plot using seaborn
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='Hours_Studied', y='Final_Score', hue='Performance', palette='viridis', s=100)
plt.title('Seaborn Scatter Plot: Hours vs Score')
plt.show()

# 2. Heatmap for correlation analysis
plt.figure(figsize=(8, 6))
# Select only numeric columns for correlation matrix
numeric_df = df.select_dtypes(include=[np.number])
corr_matrix = numeric_df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap')
plt.show()

# 3. Boxplot for categorical analysis
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='Performance', y='Final_Score', order=['Low', 'Average', 'Good'], palette='Set2')
```

```
plt.title('Boxplot: Performance vs Final Score')
plt.show()
```

Outputs :

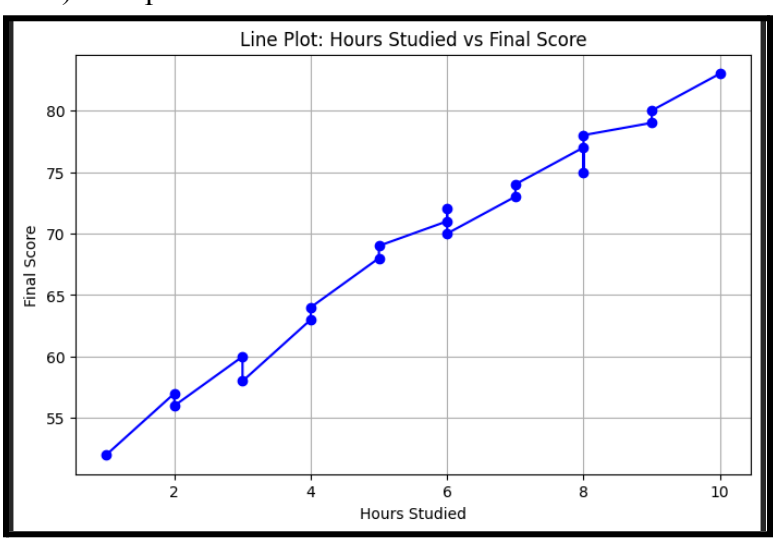
1) Numpy Basic Computations

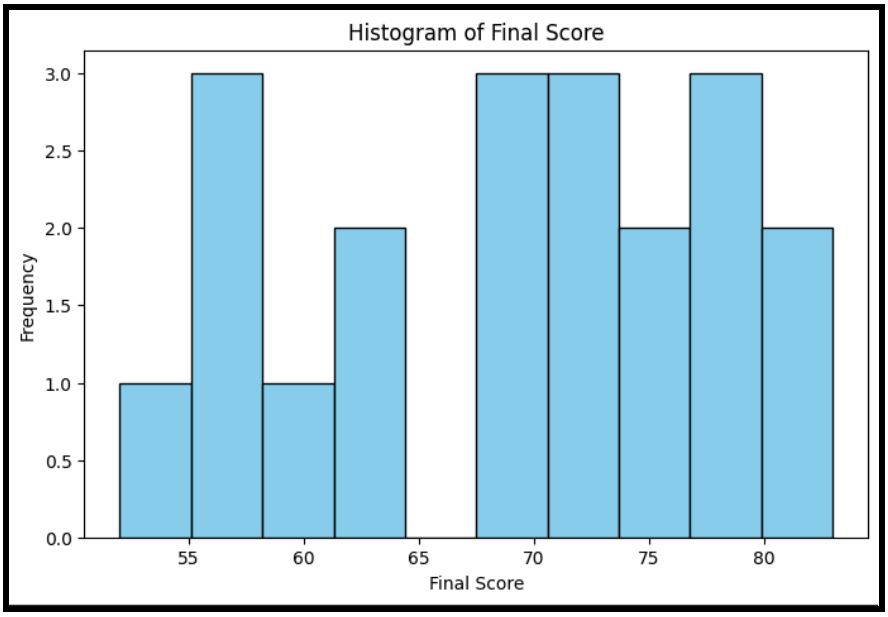
```
--- Exercise 1: NumPy Basics ---
Mean Final Score: 68.95
Median Final Score: 70.50
Standard Deviation: 8.71
Normalized Scores (first 5): [0.   0.16 0.26 0.39 0.52]
```

2) Calculate the Performance Label

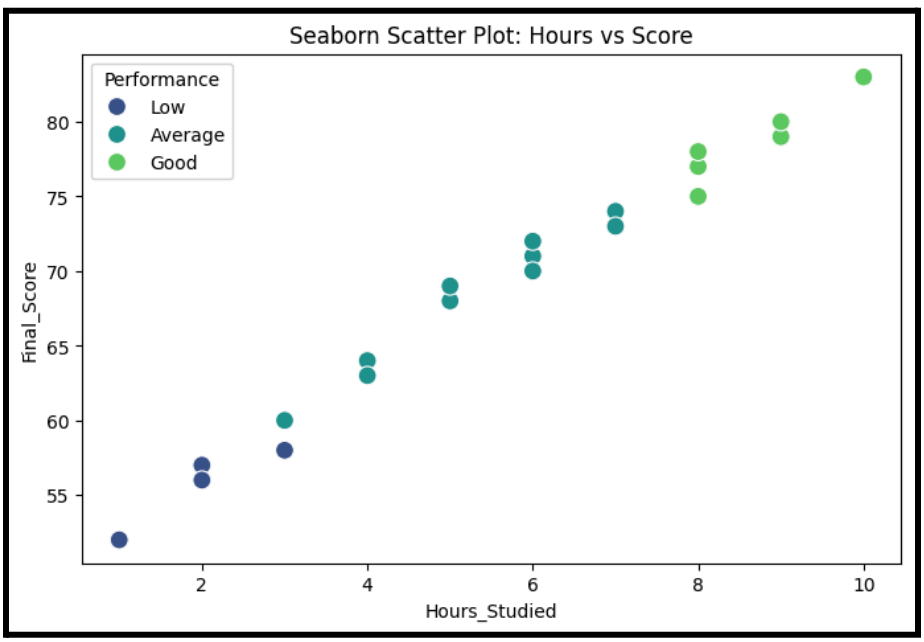
```
--- Exercise 2: Pandas Data Handling ---
Shape of dataset: (20, 5)
Columns: ['Hours_Studied', 'Attendance', 'Assignment_Score', 'Midterm_Score', 'Final_Score']
Missing Values:
 Hours_Studied       0
Attendance           0
Assignment_Score     0
Midterm_Score        0
Final_Score          0
dtype: int64

Data with Performance Label (first 5 rows):
   Final_Score Performance
0           52         Low
1           57         Low
2           60     Average
3           64     Average
4           68     Average
```

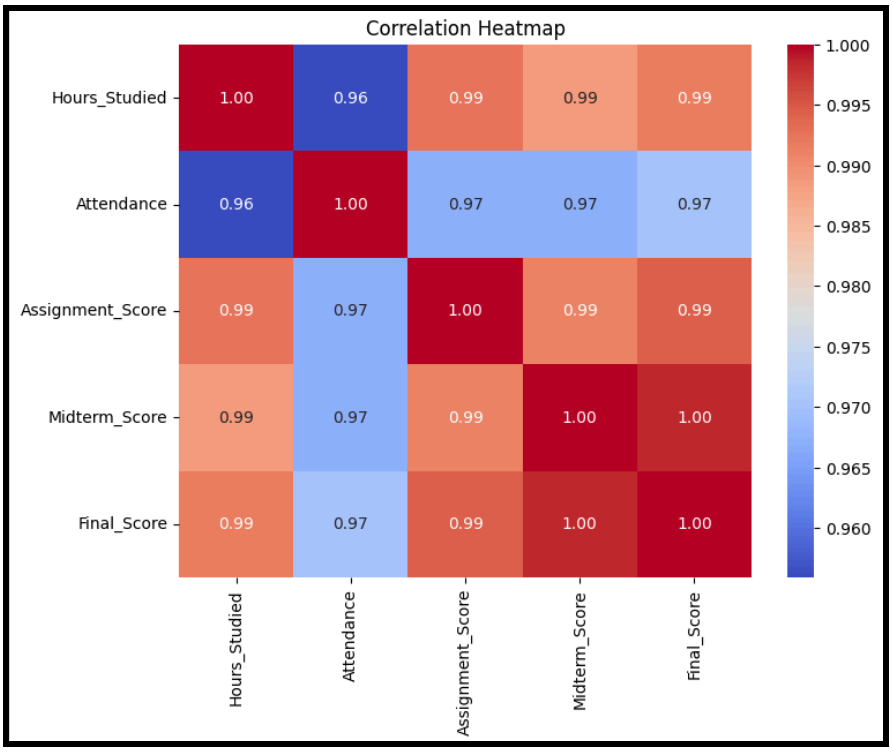
3) Matplotlib Visualization



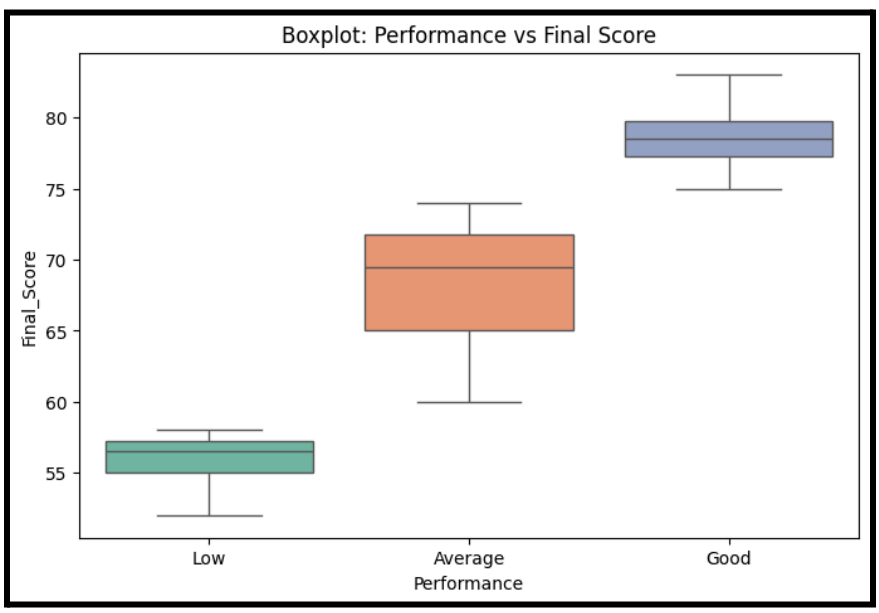


4) Seaborn Visualization





5) Boxplot Visualization



Conclusion : Hence we have successfully applied the NumPy, Pandas, Matplotlib & Seaborn for Machine Learning